

CS301 – IT Solution Architecture: Project Video Proposal (10% of total marks)

The objective of this project proposal is for the team to present an architectural design that effectively addresses both the functional and non-functional requirements specified by the customer. The team may also innovate by proposing an additional requirement for differentiation. The required deliverable is a video proposal.

1. General grading criteria for project deliverables:

a. For diagrams:

- i.* Clear and concise
- ii.* Follows recognized standards or includes an unambiguous legend.
- iii.* The system depicted meets requirements.

b. For analysis:

- i.* Clarity, completeness, and conciseness of the analysis
- ii.* Rigor and accuracy of the analysis
- iii.* Correctness and impact of recommendations
- iv.* The degree to which the tradeoff space is discussed and understood.
- v.* The specificity of the argument to the given context.

You may consult other teams or sources without penalty, but **any input beyond the course materials**, whether from **individuals or external documents or AI tools must be acknowledged** in your submission.

2. Instructions – Video Proposal

As the architect for this project, the team will present its proposed architectural design of the solution to the management, development, and operations teams. This presentation aims to demonstrate how the architecture satisfies all specified requirements.

1. Keep your video presentation to **10 minutes**.
2. Your proposal should describe the proposed solution architecture of the project that fulfills the mandatory Features 1-4 as specified in the project requirements and an additional X-factor requirement proposed by your team. The team should attempt to show screens that fulfill the functional requirements. For example, the team may choose to implement the web pages for the use cases. If time is a constraint for coding, using prototyping tools (e.g., Figma) is allowed.

3. Submit your deliverables via **eLearn->Assignment-> G{SectionNo} - Project Submissions** by the deadline due. Name your video as **G1-{TeamNo}-VideoProposal.mp4 (e.g., G1-T1-Video.mp4)**.
4. Your project must not be replicated from previous semester projects. Late submission or failure to comply with the instructions will be penalized.

3. Grading Criteria - Video Presentation

1. [5%] Conformance to the instructions.
 - [-] Missing, incomplete or late submission.
2. [30%] Video Quality and completeness of the use cases, diagrams or other notations.
 - [-] Ambiguity or inconsistency among views or missing views
 - [-] Unexplained choices, designs, services
 - [+] Clear descriptions, explanations, readable figures and tables, scannable text
 - [+] Good video presentation
3. [45%] Designs proposed for sponsor's functional and non-functional requirements.
 - [-] Missing design for functional and/or non-functional requirements.
 - [-] Wrong conceptual understanding of solution designs.
 - [-] Alternative designs not compared or not well-supported by evidence or reputable references.
 - [+] Clear plan for how designs will be evaluated (maintainability, security, performance, availability).
 - [+] Bonus features or innovative designs proposed by the team provided basic features are already covered.
4. [20%] Idea and Design proposed for the team's X-factor.
 - [-] Requirement or design is not aligned with the project.
 - [+] Requirement and design demonstrate clear potential impact for the sponsor.

4. Points to note

1. If you are proposing using existing open-source codes, you and your team members must be competent to understand the codebase, modify and deploy the codebase.
2. Java, Python, Vue.js, React.js, Spring Boot and NodeJS are commonly used but the team can choose other technologies.
3. You are required to implement this solution entirely on the AWS cloud (project constraint). Do not include services just to look comprehensive. Make sure the service is justified in your design.
4. You can also propose to implement for other areas of qualities found in [ISO 25010](#) if appropriate.
5. Your final solution can still differ from this proposed solution. Use this opportunity to validate your design and understanding of the requirements.

1. Required Information for the Video Proposal

- **GitHub Team name, team members (name and matriculation).**

- **Background and Business Needs**

Provide the background context of the project. Include your team's understanding of the problem statement. Recommend 1 slide, design should be the focus of your proposal.

- **Understanding of the Stakeholders**

Identify and document the primary stakeholders who will interact with this application. During implementation, these stakeholders will be assigned appropriate IAM users, groups, roles, and permissions within AWS during system configuration.

Stakeholder	Stakeholder Description	Permissions
<Title of this stakeholder>	<short description of this stakeholder>	<what is/are the AWS permissions required>

- **Architectural Significant Requirements (ASRs)**

Architecturally Significant Requirements (ASRs) are essential criteria that inform the design of the system's architecture. If the removal of a requirement would result in a substantial change to the overall architecture, it should be considered part of the ASRs. In relation to this project, we will also be presenting to the UBS client and must prioritize the necessary functional features.

Each team will document **3 ASRs**, as follows.

1. **(Admin's perspective)** An ASR focusing on one or more admin's functional requirements and incorporate non-functional requirements (e.g., secure login, data encrypted at rest)
2. **(Agent's perspective)** An ASR focusing on one or more agent's functional requirements and incorporate non-functional requirements (e.g., failover, response times)
3. **(X-factor)** The team can propose a new requirement related to CRM that use additional components that can differentiate your team from the rest.

For (1) and (2), descriptions should emphasize events related to functional requirements and integrate considerations of non-functional quality within the appropriate context, rather than presenting them as separate scenarios. The Admin and Agent ASRs should be derived from the mandatory Features 1–4 (e.g., user management, client onboarding, transaction ingestion). For item (3), each team is requested to distinguish itself by proposing a novel requirement that would have a tangible impact on UBS's use of the CRM.

Discuss and document each ASR following the below use case scenario format. For presentation, you can illustrate the sequence of events in your preferred mode that makes the presentation easier to understand. Documenting using sequence diagram(s) will be required for the final report but for proposal video submission, the team may also adopt other methods to better demonstrate/explain how each ASR is satisfied in the solution. For instance, utilizing screenshots

of the user interface for functional features can enhance comprehensibility and architecture diagrams with narrations on the flow of events.

Use Case Title - <short title of your architectural significant requirement>	
Use Case ID	<numbering format e.g., UC-01 for user feature 1>
Description	<short description of this use case>
Actors	<who are involved. Can be the stakeholders or software systems>
Main Flow of events	<step by step of events how the actors use or interact with your system and its architectural components>
Alternative Flow of events	<scenarios that deviate from the main flow, e.g., exceptions flow>
Pre-conditions	<Required conditions before the start of the main flow>
Post-conditions	<Expected conditions after the end of the main flow>

All AWS services and system components mentioned in the ASRs should be explicitly introduced within the Solution View section prior to their reference.

- **Solution Views**

Create diagram(s) to illustrate your system's intended architecture structure. Your architecture should comprise of key AWS components you will be implementing. To describe the AWS diagrams, provide the necessary arrows and annotation for your use cases flow of events. Avoid adding AWS components that are not important to illustrate or complicate the drawing. Remember your solution architecture diagram must be easily understood.

You may directly use the AWS icons or AWS architecture tools like Mermaid or draw.io

Solution View

All AWS services and system components presented in the Solution View should be explicitly referenced in the subsequent ASRs section. If any component is not referenced, please provide justification for its inclusion in this view.

- **Proposed Budget to develop the prototype**

You are required to estimate how much your solution costs for developing the prototype architecture in this project. The budget should reflect a design that satisfies the mandatory functional and non-functional requirements. We will compare the planned and actual costs in the final project submission.

All AWS services and components referenced in the budget must be clearly introduced in the Solution View.

Prototype Budget

Hardware/Software/ Service	Description	Cost for Production Environment
< e.g., AWS components to deploy the prototype in an actual production environment>	<Description of this item>	<e.g., AWS calculator >